

Docker-03

1. Create a image from running container

```
ubuntu@ip-172-31-10-98:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nginx:latest	5aca99593157	241MB	66MB	

```
ubuntu@ip-172-31-10-98:~$ docker run -itd 5aca99593157
872d2a60bcad44db77516a5e7404d401a7d137ca90ff5bebf20d649e7f0ace17
ubuntu@ip-172-31-10-98:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
872d2a60bcad	5aca99593157	"/docker-entrypoint..."	8 seconds ago	Up 8 seconds	80/tcp	awesome_carver

Run a Container : docker run -dit --name mynginx nginx

```
ubuntu@ip-172-31-10-98:~$ docker run -dit --name mynginx nginx
5518de0badc6bdbecfa35c33da76ba5a328bc75d67ce4ebcd6c2986d6be3f37b
ubuntu@ip-172-31-10-98:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
5518de0badc6	nginx	"/docker-entrypoint..."	4 seconds ago	Up 3 seconds	80/tcp	mynginx
872d2a60bcad	5aca99593157	"/docker-entrypoint..."	32 seconds ago	Up 31 seconds	80/tcp	awesome_carver

Make Changes Inside Container :docker exec -it mynginx bash

apt update

apt install vim -y

```
ubuntu@ip-172-31-10-98:~$ docker exec -it mynginx bash
root@5518de0badc6:/# apt update
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [195 kB]
Fetched 10.1 MB in 1s (15.4 MB/s)
5 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Create Image from Container : Create Image from Container

```
ubuntu@ip-172-31-10-98:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nginx:latest	5aca99593157	241MB	66MB	

```
ubuntu@ip-172-31-10-98:~$ docker commit mynginx custom-nginx:v1
sha256:a84660173250a1f0ae83c29cbeabf52e5006bd1cf2d72bf4fe217a9f18bed730
ubuntu@ip-172-31-10-98:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
custom-nginx:v1	a84660173250	342MB	94.6MB	
nginx:latest	5aca99593157	241MB	66MB	

```
ubuntu@ip-172-31-10-98:~$
```

2. Copy image from local machine to docker server and load the image.

Save Image on Source Machine :`docker save -o custom-nginx.tar custom-nginx:v1`

Copy Image to Docker Server: `scp custom-nginx.tar ubuntu@<docker-server-ip>:/home/ubuntu/`

```
ubuntu@ip-172-31-10-98:~$ scp -i mumbai.pem custom-nginx.tar ubuntu@13.232.139.45:/home/ubuntu
custom-nginx.tar
ubuntu@ip-172-31-10-98:~$
```

100% 90MB 155.5MB/s 00:00

Check in the docker server whether the tar file copy or not

```
ubuntu@ip-172-31-36-184:~$ ls
ubuntu@ip-172-31-36-184:~$ ls
custom-nginx.tar
ubuntu@ip-172-31-36-184:~$
```

i-06b487a7d49b8e126 (ubuntu)

PublicIPs: 13.232.139.45 PrivateIPs: 172.31.36.184

Load Image on Docker Server :`docker load -i custom-nginx.tar`

Verify : docker images

```
ubuntu@ip-172-31-36-184:~$ sudo docker load -i custom-nginx.tar
Loaded image: custom-nginx:v1
ubuntu@ip-172-31-36-184:~$ docker images
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@ip-172-31-36-184:~$ sudo docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
custom-nginx:v1	a84660173250	342MB	94.6MB	

ubuntu@ip-172-31-36-184:~\$

3. Create Docker image using alpine and customize with tomcat.

Create Dockerfile :

FROM alpine:latest

RUN apk update && \
apk add openjdk17 wget

RUN wget
<https://dlcdn.apache.org/tomcat/tomcat-10/v9.0.108/bin/apache-tomcat-9.0.108.tar.gz> && \
tar -xzf apache-tomcat-9.0.108.tar.gz && \
mv apache-tomcat-9.0.108 /opt/tomcat

EXPOSE 8080

CMD ["/opt/tomcat/bin/catalina.sh","run"]

```
ubuntu@ip-172-31-36-184:~$ cat Dockerfile
FROM alpine:latest

RUN apk update && \
    apk add openjdk17 wget

RUN wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.118/bin/apache-tomcat-9.0.118.tar.gz && \
    tar -xzf apache-tomcat-9.0.118.tar.gz && \
    mv apache-tomcat-9.0.118 /opt/tomcat

EXPOSE 8080

CMD ["/opt/tomcat/bin/catalina.sh","run"]
```

Build Image :docker build -t alpine-tomcat:v1 .

```
ubuntu@ip-172-31-36-184:~$ sudo docker build -t alpine-tomcat:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  94.65MB
Step 1/5 : FROM alpine:latest
latest: Pulling from library/alpine
6a0ac1617861: Pulling fs layer
6a0ac1617861: Download complete
6a0ac1617861: Pull complete
a63813f19ccb: Download complete
4e0cb4ac63e7: Download complete
Digest: sha256:5b10f432ef3dalb8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
Status: Downloaded newer image for alpine:latest
--> 5b10f432ef3d
Step 2/5 : RUN apk update &&          apk add openjdk17 wget
--> Running in 54229734e2bd
v3.23.4-330-g0ff1e5c4d6c [https://dl-cdn.alpinelinux.org/alpine/v3.23/main]
```

```
--> Removed intermediate container 61c40a6d40fc
--> 3aa0346ab901
Step 4/5 : EXPOSE 8080
--> Running in 185fe50feda8
--> Removed intermediate container 185fe50feda8
--> 8f11fb330060
Step 5/5 : CMD ["/opt/tomcat/bin/catalina.sh","run"]
--> Running in ae8ea2872c1e
--> Removed intermediate container ae8ea2872c1e
--> a850a225efce
Successfully built a850a225efce
Successfully tagged alpine-tomcat:v1
```

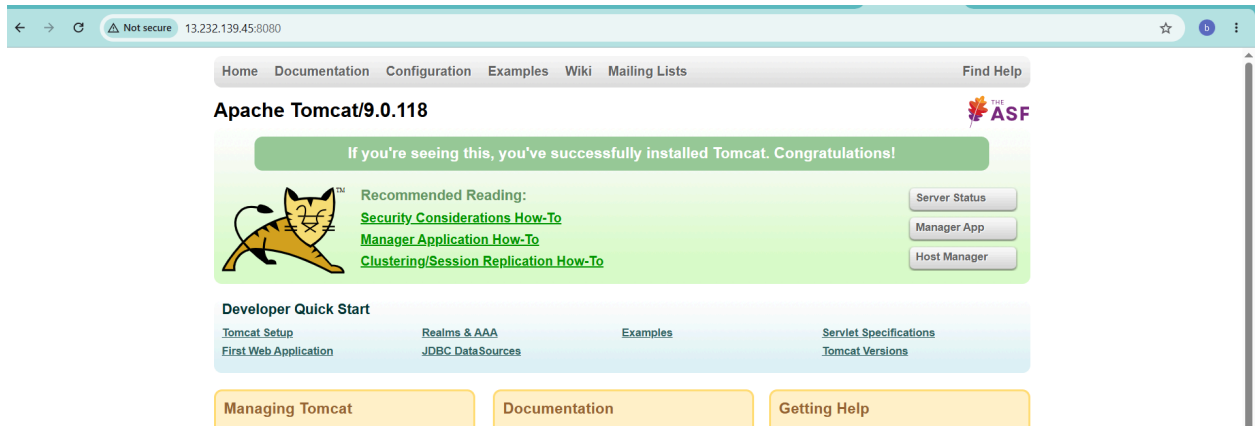
Run Container :docker run -d -p 8080:8080 alpine-tomcat:v1

```
ubuntu@ip-172-31-36-184:~$ docker run -d -p 8080:8080 alpine-tomcat:v1
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@ip-172-31-36-184:~$ sudo docker run -d -p 8080:8080 alpine-tomcat:v1
f86877c1d081e899afc7f7d0a20d35c3c7618de69e18ead3f7f9a979ea3f4030
ubuntu@ip-172-31-36-184:~$ docker ps
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
ubuntu@ip-172-31-36-184:~$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
f86877c1d081	alpine-tomcat:v1	"/opt/tomcat/bin/cat..."	4 minutes ago	Up 4 minutes	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	upbeat_beaver

i-06b487a7d49b8e126 (ubuntu)

verify:



4. Create single stage and multi stage docker file using this source code:<https://github.com/betawins/multi-stage-example.git>

Clone Repository :git clone

<https://github.com/betawins/multi-stage-example.git>

cd multi-stage-example

Single Stage Dockerfile

FROM maven:3.9.6-eclipse-temurin-17

WORKDIR /app

COPY . .

RUN mvn clean package

EXPOSE 8080

CMD ["java","-jar","target/*.jar"]

```
ubuntu@ip-172-31-10-98:~/multi-stage-example$ cat Dockerfile
FROM maven:3.9.6-eclipse-temurin-17

WORKDIR /app

COPY . .

RUN mvn clean package

EXPOSE 8080

CMD ["java","-jar","target/*.jar"]
ubuntu@ip-172-31-10-98:~/multi-stage-example$
```

docker build -t springboot-single:v1 .

```
ubuntu@ip-172-31-10-98:~/multi-stage-example$ docker build -t springboot-single:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 209.4kB
Step 1/6 : FROM maven:3.9.6-eclipse-temurin-17
----> 29a1658b1f30
Step 2/6 : WORKDIR /app
----> Using cache
----> a3f411a9af94
Step 3/6 : COPY . .
----> Using cache
----> d847a7b34d23
Step 4/6 : RUN mvn clean package
----> Using cache
----> 47d46f41462f
Step 5/6 : EXPOSE 8080
----> Using cache
----> 82330ed40bd1
Step 6/6 : CMD ["java","-jar","target/*.jar"]
----> Using cache
----> b578b94a53fa
Successfully built b578b94a53fa
Successfully tagged springboot-single:v1
```

Verify :docker images

```
ubuntu@ip-172-31-10-98:~/multi-stage-example$ docker images
\
e


| IMAGE                          | ID           | DISK USAGE | CONTENT SIZE | EXTRA |
|--------------------------------|--------------|------------|--------------|-------|
| custom-nginx:v1                | a84660173250 | 342MB      | 94.6MB       |       |
| maven:3.9.6-eclipse-temurin-17 | 29a1658b1f30 | 728MB      | 221MB        |       |
| nginx:latest                   | 5aca99593157 | 241MB      | 66MB         | U     |
| springboot-single:v1           | b578b94a53fa | 871MB      | 283MB        |       |


ubuntu@ip-172-31-10-98:~/multi-stage-example$ \
```

Multi Stage Dockerfile :

Build Stage

FROM maven:3.9.6-eclipse-temurin-17 AS builder

WORKDIR /app

COPY . .

RUN mvn clean package

Runtime Stage

FROM eclipse-temurin:17-jre

WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java","-jar","app.jar"]

```

ubuntu@ip-172-31-10-98:~/multi-stage-example$ cat Dockerfile
# Build Stage
FROM maven:3.9.6-eclipse-temurin-17 AS builder

WORKDIR /app

COPY . .

RUN mvn clean package

# Runtime Stage
FROM eclipse-temurin:17-jre

WORKDIR /app

COPY --from=builder /app/target/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
ubuntu@ip-172-31-10-98:~/multi-stage-example$ █

```

Build :`docker build -t springboot-multi:v1 .`

```

ubuntu@ip-172-31-10-98:~/multi-stage-example$ docker build -t springboot-multi:v1 .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 209.4kB
Step 1/9 : FROM maven:3.9.6-eclipse-temurin-17 AS builder
--> 29a1658b1f30
Step 2/9 : WORKDIR /app
--> Using cache
--> a3f411a9af94
Step 3/9 : COPY . .
--> Using cache
--> 377af5aa210b
Step 4/9 : RUN mvn clean package
--> Using cache
--> 998d98ae13ca
Step 5/9 : FROM eclipse-temurin:17-jre
--> 0d79988c6879
Step 6/9 : WORKDIR /app
--> Using cache
--> cd2477c0f020
Step 7/9 : COPY --from=builder /app/target/*.jar app.jar
--> Using cache
--> 5c6f947cd7c0
Step 8/9 : EXPOSE 8080
--> Using cache
--> 40159b1e7719
Step 9/9 : ENTRYPOINT ["java", "-jar", "app.jar"]

```


Run :docker run -d -p 8080:8080 springboot-multi:v1

```
ubuntu@ip-172-31-10-98:~/multi-stage-example$ \docker run -d -p 8080:8080 springboot-multi:v1
9ece932464d363e4348b6a7b17bbfa5b715f94350dd42b2e0cb0fd7e53025bcf
ubuntu@ip-172-31-10-98:~/multi-stage-example$ docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED        STATUS        PORTS                               NAMES
9ece932464d3   springboot-multi:v1 "java -jar app.jar"      11 seconds ago Up 11 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   silly_kepler
5518de0badc6   nginx              "/docker-entrypoint..." 6 hours ago   Up 6 hours   80/tcp                             mynginx
872d2a60bcad   5aca99593157       "/docker-entrypoint..." 6 hours ago   Up 6 hours   80/tcp                             awesome_carver
ubuntu@ip-172-31-10-98:~/multi-stage-example$ \
```

Verify : docker images

```
ubuntu@ip-172-31-10-98:~/multi-stage-example$ docker images
\
e
IMAGE                                ID                                DISK USAGE  CONTENT SIZE  EXTRA
custom-nginx:v1                     a84660173250                     342MB       94.6MB
eclipse-temurin:17-jre              0d79988c6879                     430MB       114MB
maven:3.9.6-eclipse-temurin-17     29a1658b1f30                     728MB       221MB
nginx:latest                         5aca99593157                     241MB       66MB      U
springboot-multi:v1                 a4f3cfb0c0f9                     462MB       126MB
springboot-single:v1                b578b94a53fa                     871MB       283MB
ubuntu@ip-172-31-10-98:~/multi-stage-example$ \
```

i-08ba2672d4332489d (docker-test)

Verify : Public ip: 8080



Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Wed Jun 03 09:20:00 UTC 2026
There was an unexpected error (type=Not Found, status=404).
No message available

5. Install docker compose and execute sample application.

Install Docker Compose :sudo apt update

sudo apt install docker-compose-v2 -y

```
ubuntu@ip-172-31-10-98:~$ sudo apt install docker-compose-v2 -y
Installing:
  docker-compose-v2

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 3
  Download size: 14.8 MB
  Space needed: 66.6 MB / 43.9 GB available

Get:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu/resolute/universe amd64v3 docker-compose-v2 amd64 2.40.3+ds1-0ubuntu1 [14.8
Fetched 14.8 MB in 0s (99.8 MB/s)
Selecting previously unselected package docker-compose-v2.
(Reading database ... 134294 files and directories currently installed.)
Preparing to unpack .../docker-compose-v2_2.40.3+ds1-0ubuntu1_amd64v3.deb ...
Unpacking docker-compose-v2 (2.40.3+ds1-0ubuntu1) ...
Setting up docker-compose-v2 (2.40.3+ds1-0ubuntu1) ...
Scanning processes...
Scanning linux images...

Pending kernel upgrade!
Running kernel version:
  7.0.0-1004-aws
Diagnostics:
  The currently running kernel version is not the expected kernel version 7.0.0-1006-aws.
```

docker-compose.yml :

version: '3'

services:

web:

image: nginx

ports:

- "80:80"

```
ubuntu@ip-172-31-10-98:~$ vi docker-compose.yml
ubuntu@ip-172-31-10-98:~$ cat docker-compose.yml
version: '3'

services:
  web:
    image: nginx
    ports:
      - "80:80"
```

Run Application :docker compose up -d

```
ubuntu@ip-172-31-10-98:~$ docker compose up -d
WARN[0000] /home/ubuntu/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 2/2
 ✓ Network ubuntu_default Created
 ✓ Container ubuntu-web-1 Started
ubuntu@ip-172-31-10-98:~$ docker compose ps
WARN[0000] /home/ubuntu/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
ubuntu-web-1	nginx	"/docker-entrypoint..."	web	11 seconds ago	Up 11 seconds	0.0.0.0:80->80/tcp, [::]:80->80/tcp

```
ubuntu@ip-172-31-10-98:~$
```

verify:



Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

6. Implement solution to scan images when pushed to docker registry.

Docker login:

```
ubuntu@ip-172-31-10-98:~$ docker login

USING WEB-BASED LOGIN

Info → To sign in with credentials on the command line, use 'docker login -u <username>'

Your one-time device confirmation code is: BMTR-DSJJ
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser...

WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
ubuntu@ip-172-31-10-98:~$ docker tag nginx:alpine2 bhargav0117/nginx-alpine2
```

Docker images:

```
ubuntu@ip-172-31-10-98:~$ docker images
```

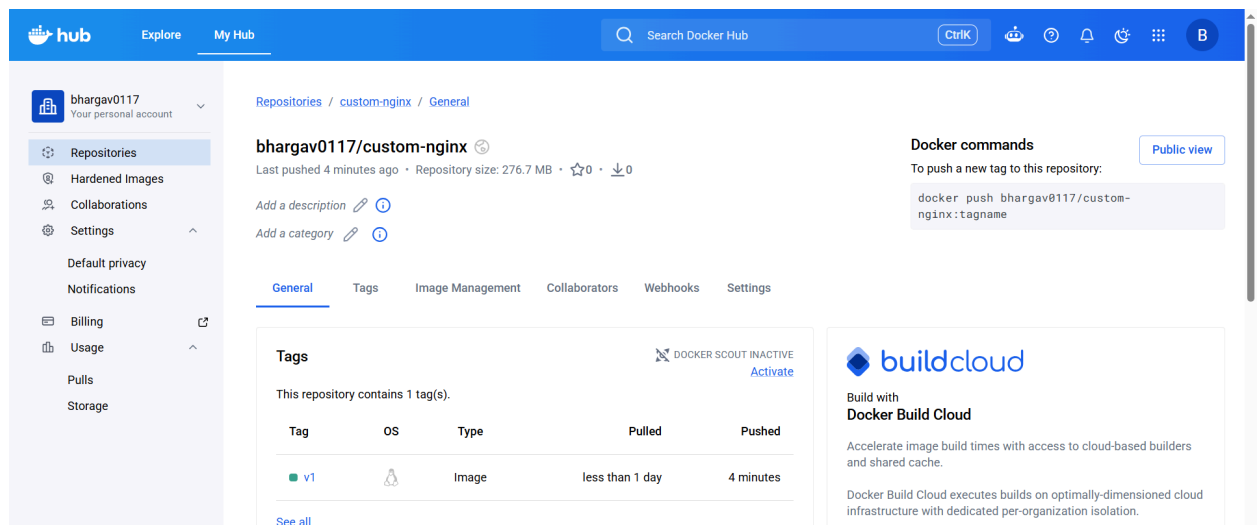
IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
custom-nginx:v1	a84660173250	342MB	94.6MB	
eclipse-temurin:17-jre	0d79988c6879	430MB	114MB	
maven:3.9.6-eclipse-temurin-17	29a1658b1f30	728MB	221MB	
nginx:latest	5aca99593157	241MB	66MB	U
springboot-multi:v1	a4f3cfb0c0f9	462MB	126MB	U
springboot-single:v1	b578b94a53fa	871MB	283MB	

```
ubuntu@ip-172-31-10-98:~$ docker tag custom-nginx:v1 bhargav0117/custom-nginx:v1
```

Push to the docker hub

```
ubuntu@ip-172-31-10-98:~$ docker push bhargav0117/custom-nginx:v1
The push refers to repository [docker.io/bhargav0117/custom-nginx]
4839851a7db0: Pushed
5b4d6ff92fc4: Mounted from library/nginx
830625e1ac85: Mounted from library/nginx
5431d0092ffd: Mounted from library/nginx
b4a248c845e5: Mounted from library/nginx
7f8b1a2b17d8: Mounted from library/nginx
45381ecb0e2f: Mounted from library/nginx
13fd728be9eb: Mounted from library/nginx
v1: digest: sha256:a84660173250a1f0ae83c29cbeabf5265006bd1cf2d72bf4fe217a9f18bed730 size: 2042
ubuntu@ip-172-31-10-98:~$
```

Verify in docker hud



The screenshot shows the Docker Hub interface for the repository `bhargav0117/custom-nginx`. The page includes a sidebar with navigation options like `Repositories`, `Hardened Images`, `Collaborations`, `Settings`, `Billing`, `Usage`, `Pulls`, and `Storage`. The main content area displays the repository details, including the tag `v1` and the Docker commands to push a new tag. The `Tags` section shows the tag `v1` with a pulled time of less than 1 day and a pushed time of 4 minutes. The `Docker commands` section shows the command `docker push bhargav0117/custom-nginx:tagname`.

Click on repository→click on settings→enable docker scout scanning

Features and controls that help you uncover, understand, and fix issues with your container images in Docker Hub

Upgrade

- ☒ **Docker Scout image analysis** RECOMMENDED
Know when new CVEs impact your images, learn where they're introduced, and get recommendations for remediation options. [Enable repos in bulk on Scout Dashboard](#) ↗
- ☐ **Static scanning**
Images will be scanned once when pushed and the vulnerability report saved at that point in time.
- ☐ **None**

bhargav0117
Your personal account

- Repositories
- Hardened Images
- Collaborations
- Settings
- Default privacy
- Notifications
- Billing
- Usage
- Pulls
- Storage

Repositories / custom-nginx / Tags / v1

bhargav0117/custom-nginx:v1

[View recommended base image fixes](#)

MANIFEST DIGEST sha256:a84660173250a1f0ae83c29ceabf5265006bd1cf2d72bf4fe217a9f18bed730

OS/ARCH
linux/amd64

COMPRESSED SIZE
90.22 MB

LAST PUSHED
7 minutes by bhargav0117

TYPE
Image

VULNERABILITIES

2	9	23	65	1
---	---	----	----	---

MANIFEST DIGEST
sha256:a8466017...

Image Layers

Specifications

Layers (19)

Layer	CVE Count	Size
nginx:1	2 8 11 61 0	
bhargav0117/custom-nginx...	0 1 12 4 1	
18 nginx -g daemon off;		31.51 MB

Vulnerabilities (100) Packages (241)

Give feedback
Analyzed by

☒ Fixable
 ☐ Show excepted

CVE ID	Severity	Fixable	Present In	Affected
--------	----------	---------	------------	----------

[Reset filters](#)

Aws configure:

```
ubuntu@ip-172-31-10-98:~$ aws --version
aws-cli/2.31.35 Python/3.14.4 Linux/7.0.0-1004-aws source/x86_64.ubuntu.26
ubuntu@ip-172-31-10-98:~$ aws configure
AWS Access Key ID [None]: AKIA5NQAPM2ETWSY4G54
AWS Secret Access Key [None]: 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
Default region name [None]: ap-south-1
Default output format [None]: json
```

Create Repository:

aws ecr create-repository \

--repository-name demo-app \

--image-scanning-configuration scanOnPush=true

```
ubuntu@ip-172-31-10-98:~$ aws ecr create-repository \
--repository-name demo-app \
--image-scanning-configuration scanOnPush=true
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:ap-south-1:922345236105:repository/demo-app",
    "registryId": "922345236105",
    "repositoryName": "demo-app",
    "repositoryUri": "922345236105.dkr.ecr.ap-south-1.amazonaws.com/demo-app",
    "createdAt": "2026-06-03T10:19:20.863000+00:00",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": true
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}
```

verify

```
ubuntu@ip-172-31-10-98:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
922345236105.dkr.ecr.ap-south-1.amazonaws.com/demo-app:v1	a4f3cfb0c0f9	462MB	126MB	U
bhargav0117/custom-nginx:v1	a84660173250	342MB	94.6MB	
custom-nginx:v1	a84660173250	342MB	94.6MB	
eclipse-temurin:17-jre	0d79988c6879	430MB	114MB	
maven:3.9.6-eclipse-temurin-17	29a1658b1f30	728MB	221MB	
nginx:latest	5aca99593157	241MB	66MB	U
springboot-multi:v1	a4f3cfb0c0f9	462MB	126MB	U
springboot-single:v1	b578b94a53fa	871MB	283MB	

```
ubuntu@ip-172-31-10-98:~$ docker tag app:v1 922345236105.dkr.ecr.ap-south-1.amazonaws.com/demo-app:v1
Error response from daemon: No such image: app:v1
ubuntu@ip-172-31-10-98:~$ docker tag custom-nginx:v1 922345236105.dkr.ecr.ap-south-1.amazonaws.com/demo-app:v1
```

Push Image :

docker tag app:v1

<account-id>.dkr.ecr.ap-south-1.amazonaws.com/demo-app:v1

docker push

<account-id>.dkr.ecr.ap-south-1.amazonaws.com/demo-app:v1

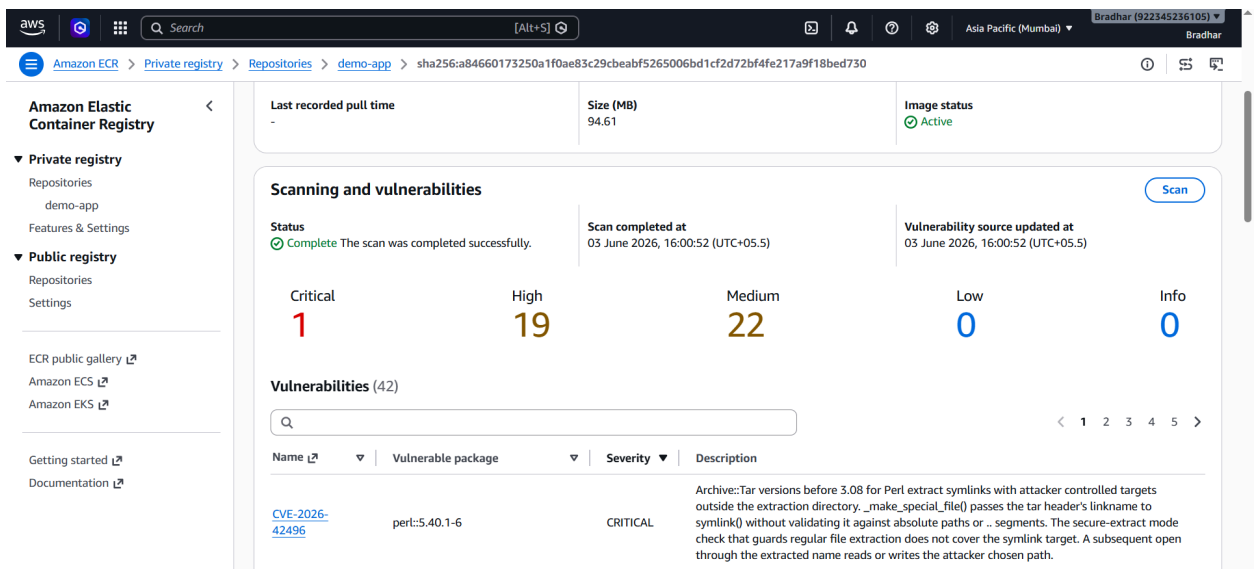
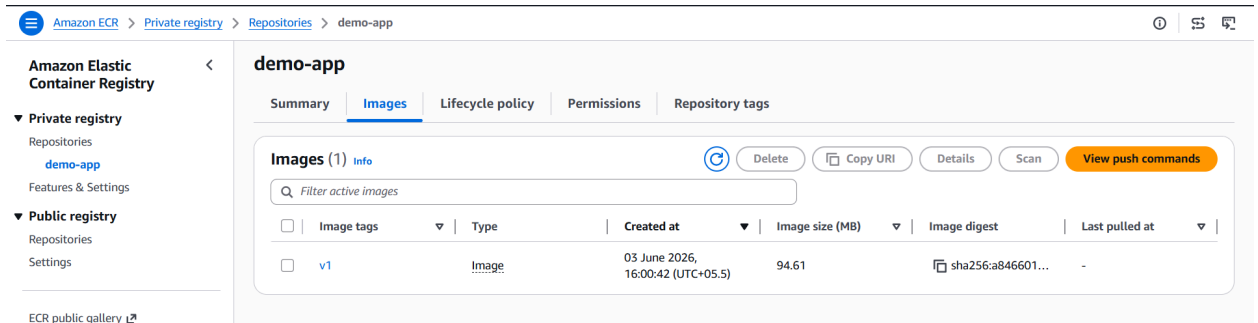
```
ubuntu@ip-172-31-10-98:~$ aws ecr get-login-password --region ap-south-1 | \
docker login --username AWS --password-stdin \
922345236105.dkr.ecr.ap-south-1.amazonaws.com

WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
```

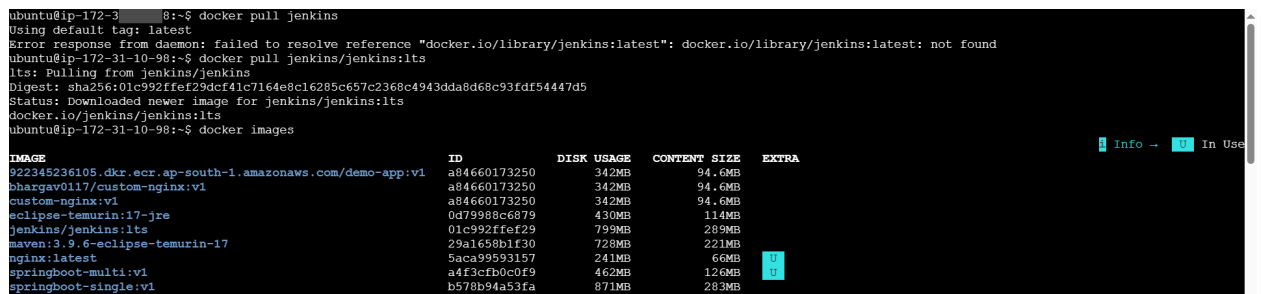
```
ubuntu@ip-172-31-10-98:~$ docker push 922345236105.dkr.ecr.ap-south-1.amazonaws.com/demo-app:v1
The push refers to repository [922345236105.dkr.ecr.ap-south-1.amazonaws.com/demo-app]
5b4d6ff92fc4: Pushed
45381ecb0e2f: Pushed
13fd728be9eb: Pushed
830625e1ac85: Pushed
5431d0092ffd: Pushed
4839851a7db0: Pushed
b4a248c845e5: Pushed
7f8b1a2b17d8: Pushed
v1: digest: sha256:a84660173250a1f0ae83c29cbeabf5265006bd1cf2d72bf4fe217a9f18bed730 size: 2042
ubuntu@ip-172-31-10-98:~$
```

AWS automatically scans the image after push:



8. Create a Jenkins pipeline to create a docker image and push the image to Docker hub

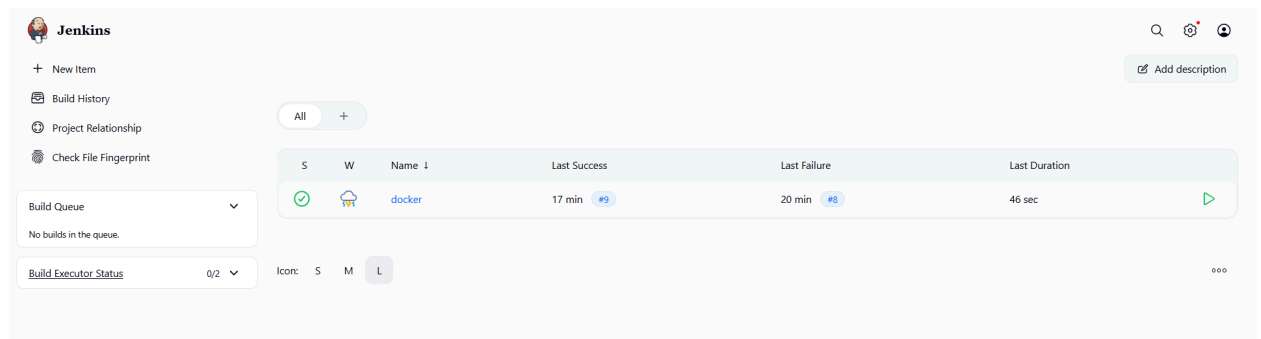
Pull jenkins: docker pull jenkins/jenkins:its



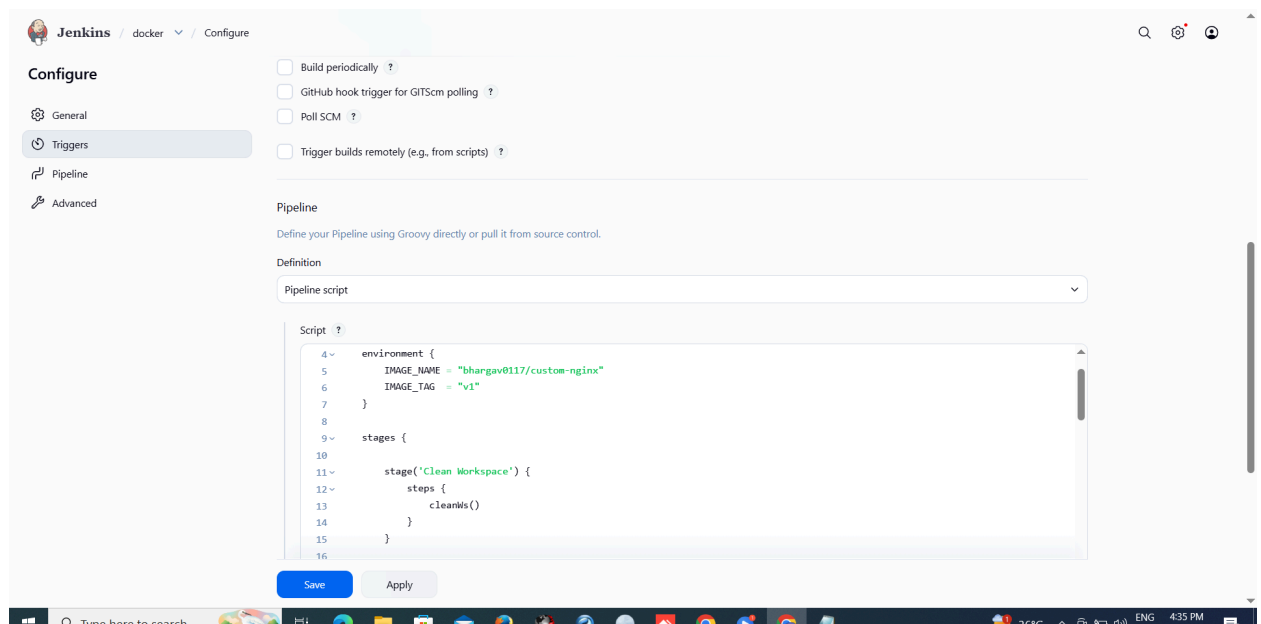
Run the container:

```
ubuntu@ip-172-31-10-98:~$ docker run -d \
--name jenkins \
-p 8080:8080 \
-p 50000:50000 \
-v jenkins_home:/var/jenkins_home \
-v /var/run/docker.sock:/var/run/docker.sock \
-v /usr/bin/docker:/usr/bin/docker \
jenkins/jenkins:lts
fbddd6a2be9ec922e220135abd50d5d8d94b8f09918ad681559772ce2ae13cfa
```

Login jenkins:



The screenshot shows the Jenkins dashboard. On the left, there's a sidebar with links: New Item, Build History, Project Relationship, and Check File Fingerprint. The main area displays a table of builds. The table has columns: S (Status), W (Webhook), Name, Last Success, Last Failure, and Last Duration. A single build is listed with the name 'docker', a status of 'Success' (green checkmark), and a duration of '46 sec'. Below the table, there's a 'Build Queue' section showing 'No builds in the queue' and a 'Build Executor Status' section showing '0/2'.



The screenshot shows the Jenkins configuration page for the 'docker' job. The left sidebar has tabs: General, Triggers, Pipeline, and Advanced. The 'Triggers' tab is selected. The main area shows the 'Pipeline' section with a 'Definition' dropdown set to 'Pipeline script'. Below this is a text area for the pipeline script, which contains the following Groovy code:

```
4 ~ environment {
5 ~     IMAGE_NAME = "bhargav0117/custom-nginx"
6 ~     IMAGE_TAG = "v1"
7 ~ }
8 ~
9 ~ stages {
10 ~
11 ~     stage('Clean Workspace') {
12 ~         steps {
13 ~             cleanWs()
14 ~         }
15 ~     }
16 ~ }
```

At the bottom of the page, there are 'Save' and 'Apply' buttons. The Windows taskbar is visible at the bottom of the screen.

Pipeline:

```
pipeline {
    agent any
```

```
environment {  
    IMAGE_NAME = "bhargav0117/custom-nginx"  
    IMAGE_TAG = "v1"  
}
```

```
stages {
```

```
    stage('Clean Workspace') {  
        steps {  
            cleanWs()  
        }  
    }  
}
```

```
    stage('Checkout') {  
        steps {  
            git branch: 'master',  
            url:  
'https://github.com/Radharapubhargav/multi-stage-example.git'  
        }  
    }  
}
```

```
    stage('Build Docker Image') {  
        steps {  
            sh ""  
            set -e  
            docker build -t $IMAGE_NAME:$IMAGE_TAG .  
            ""  
        }  
    }  
}
```

```

    }

    stage('Docker Login') {
        steps {
            withCredentials([
                usernamePassword(
                    credentialsId: 'docker',
                    usernameVariable: 'DOCKER_USER',
                    passwordVariable: 'DOCKER_PASS'
                )
            ]) {
                sh """
                    echo $DOCKER_PASS | docker login -u
$DOCKER_USER --password-stdin
                """
            }
        }
    }

    stage('Push Image') {
        steps {
            sh """
                docker push $IMAGE_NAME:$IMAGE_TAG
            """
        }
    }
}

```

```
post {
    success {
        echo '✅ Docker image built and pushed successfully'
    }
    failure {
        echo '❌ Pipeline failed - check logs'
    }
}
}
```

Save & build

The image shows the Jenkins web interface for a job named 'docker'. The build status is 'Success' (green checkmark) for build #9, dated 4 Jun 2026, 10:47:13. The left sidebar contains navigation links: Status, Changes, Console Output, Edit Build Information, Delete build '#9', Timings, Git Build Data, Pipeline Overview, Restart from Stage, Replay, Pipeline Steps, Workspaces, and Previous Build. The main content area shows that the build was started by user 'admin'. It lists the time spent: 11 ms waiting, 46 sec build duration, and 46 sec total from scheduled to completion. The Git build data shows the revision as 7006266809c41a875f61d80658c10de92a97437c and the repository as https://github.com/Radharapubhargav/multi-stage-example.git, with the ref refs/remotes/origin/master. The console output shows 'No changes.'

Jenkins / docker / #9

Status ✅ #9 (4 Jun 2026, 10:47:13) 🔍 ⚙️ 👤

📄 Add description Keep this build forever

📄 Changes

📄 Console Output

📄 Edit Build Information

🗑️ Delete build '#9'

🕒 Timings

🔗 Git Build Data

🔗 Pipeline Overview

🔄 Restart from Stage

🔄 Replay

📄 Pipeline Steps

📄 Workspaces

🔍 Previous Build

🕒 Started by user [admin](#)

🕒 This run spent:

- 11 ms waiting;
- 46 sec build duration;
- 46 sec total from scheduled to completion.

🔗 git **Revision:** 7006266809c41a875f61d80658c10de92a97437c
Repository: <https://github.com/Radharapubhargav/multi-stage-example.git>

- refs/remotes/origin/master

📄 No changes.

Started 19 min ago
Took 46 sec

Console output:

